

Trekking Management Application

Name: Oum Gupta

Roll Number: 24f3003029

Email: 24f3003029@ds.study.iitm.ac.in

Problem Statement

The project is a web based trekking management system where three different kinds of people use the same site. An admin manages the treks and approves the trek staff, the trek staff manage the treks given to them, and normal users search for treks and book their slots. The main problem the application solves is keeping the slot count correct so that a trek never gets more bookings than the number of seats available, while still letting each role do only the work that belongs to that role.

Approach

I started by deciding the three tables that the whole application needs, which are User, Trek and Booking. Instead of making separate tables for admin, staff and user I kept one User table with a `role` column, because all three log in the same way and only their permissions change. This kept the login code in one place.

After the tables I wrote the routes role by role. First the login and registration, then the admin side, then the staff side, and last the user booking side. Every protected route checks `session['role']` at the top and sends the person back to the login page if the role does not match. The slot logic is kept inside the booking route so that the available slots are reduced at the same time the booking row is created.

The database is created programmatically by `db.create_all()` and the admin account is inserted on the first run if it does not already exist, so no manual database work is needed.

Frameworks and Libraries Used

Library	Purpose
Flask	Routing and the web server
Flask-SQLAlchemy	Models and database queries
Jinja2	HTML templates with inheritance from base.html
Bootstrap 5 (CDN)	Layout, tables, cards and forms
Werkzeug security	Password hashing
SQLite	Database, created programmatically

No JavaScript is used anywhere. All the searching, filtering and form handling is done on the server side.

Data Flow

There is no API layer in this project. The frontend is HTML that Flask renders on the server and sends already finished, so every interaction is a full page load and no JSON or AJAX is involved.

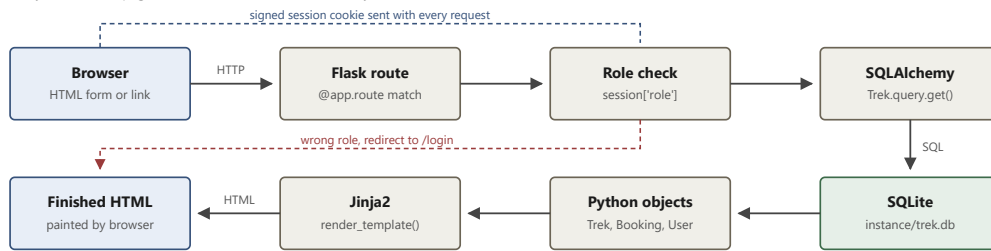
A request arrives over HTTP, Flask matches it to a route, the route checks `session['role']` and redirects to the login page if the role is wrong, then queries SQLite through SQLAlchemy. The Python objects that come back are passed to `render_template()`, Jinja2 fills the placeholders, and the finished HTML goes back to the browser.

Data reaches the backend only through HTML forms. A POST form sends its fields in the body and the route reads them with `request.form`, while the search form uses GET so the values arrive in the URL and are read with `request.args`. Identity is kept in a session cookie signed with `SECRET_KEY`, which the browser returns on every request and which cannot be forged without the key.

Every route that changes data answers with `redirect()` instead of rendering, so refreshing the page after a booking cannot book the same trek twice. Messages survive that redirect through `flash()`.

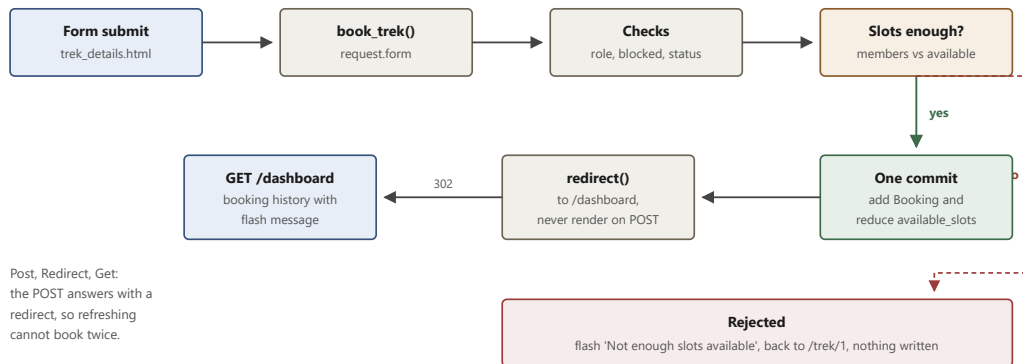
A. Request lifecycle

Every click is a full page load. No JSON, no AJAX, no API layer.

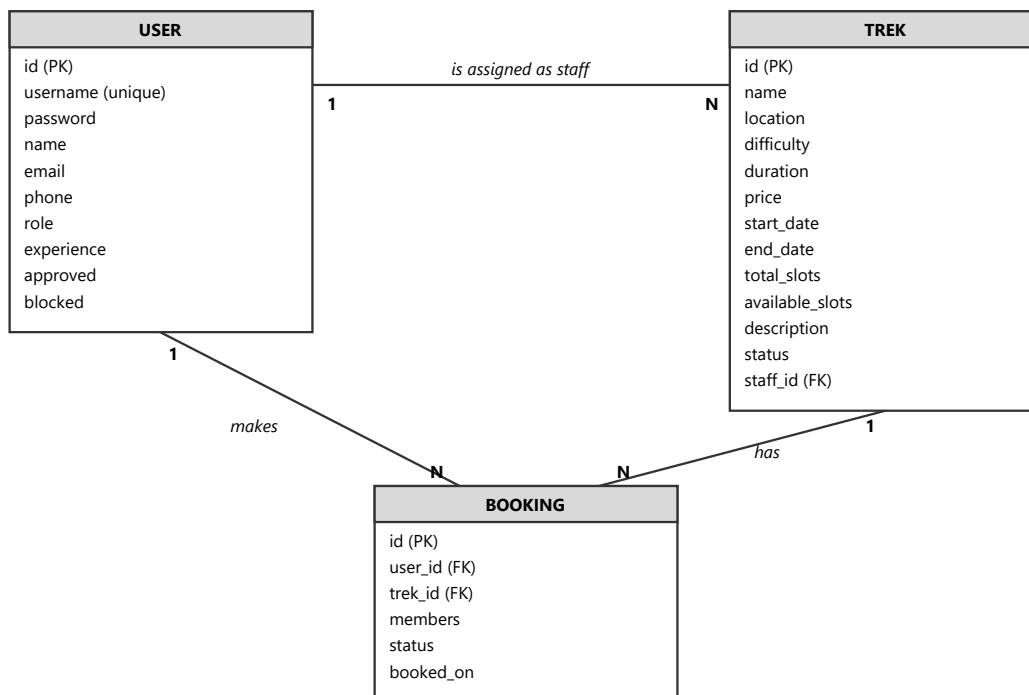


B. Booking a trek, step by step

POST /book/1 with members=2



ER Diagram



Relationships

Relationship	Type	Description
User to Booking	One to Many	One user can book many treks, each booking belongs to one user

Trek to Booking	One to Many	One trek can have many bookings
User (staff) to Trek	One to Many	One staff can be assigned many treks, a trek has at most one staff

The `role` column decides whether a User row is an admin, a staff or a trekker. `approved` is used only for staff and stays False until the admin approves the request. `blocked` is used by the admin to blacklist either a user or a staff.

API Endpoints

Authentication and Public Pages

Endpoint	Method	Access	Description
/	GET	All	Home page listing treks, supports <code>?q=</code> and <code>?difficulty=</code>
/login	GET, POST	All	Login, redirects by role, blocks unapproved or blacklisted accounts
/register	GET, POST	Public	Trekker self registration
/staff/register	GET, POST	Public	Trek staff registration, waits for admin approval
/logout	GET	Logged in	Clears the session
/trek/<tid>	GET	All	Trek details page with the booking form

Admin

Endpoint	Method	Description
/admin	GET	Dashboard with the summary counts and the full trek list
/admin/trek/add	GET, POST	Create a new trek
/admin/trek/edit/<tid>	GET, POST	Edit a trek, refuses if slots go below booked seats
/admin/trek/delete/<tid>	GET	Delete a trek along with its bookings
/admin/trek/assign/<tid>	GET, POST	Assign an approved staff to a trek
/admin/staff	GET	Pending requests and the approved staff list
/admin/staff/approve/<uid>	GET	Approve a staff request
/admin/staff/reject/<uid>	GET	Reject and remove a staff request
/admin/users	GET	List of all registered trekkers, searchable by name or ID
/admin/bookings	GET	Every booking in the system, searchable
/admin/block/<uid>	GET	Block or unblock a user or a staff

Trek Staff

Endpoint	Method	Description
/staff	GET	Treks assigned to the logged in staff
/staff/trek/<tid>	GET, POST	Update slots and status, view the participant list
/staff/booking/<bid>/cancel	GET	Remove a participant from an assigned trek

Trekker

Endpoint	Method	Description
/dashboard	GET	Booked treks with trek status, and the treks open for booking
/profile	GET, POST	Edit own name, email, phone and password
/book/<tid>	POST	Book slots on a trek
/cancel/<bid>	GET	Cancel a booking and return the slots

Features Implemented

Feature	How it works
Overbooking prevention	The booking route rejects a request when the members asked for exceed <code>available_slots</code> , and reduces the slots in the same commit as the booking row. A user cannot hold two active bookings on one trek, and booking is refused unless the trek status is Open.
Safe slot editing	When the admin or the staff changes the total slots, the seats already booked are calculated first and any smaller value is refused. Available slots are recalculated, never overwritten.
Role based access	Every protected route checks the role held in the session. A staff member can open only the treks assigned to them, checked by comparing <code>trek.staff_id</code> with the logged in staff id.
Staff approval	A staff account is created with <code>approved = False</code> and login is refused until the admin approves it. Only approved and unblocked staff appear in the assignment dropdown.
Blacklisting	A blocked user or staff cannot log in, and a blocked user cannot book.
Booking history	Cancelling changes the status to Cancelled and returns the seats instead of deleting the row. Marking a trek Completed moves all its active bookings to Completed.
Profile and photo	Every role can edit their own name, email, phone and password, and upload a profile picture which is shown as a round avatar in the navigation bar. Uploads are restricted to png, jpg, jpeg and gif under 2 MB, renamed after the user id, and users without a photo get a circle with their initial.
Search and filter	Treks are searched by name and location with a LIKE query and filtered by difficulty. The admin can also search treks, staff, users and bookings by name or ID. All of it runs in Flask with no JavaScript.

Trek status flow. A trek moves through Pending, Approved, Open, Closed and Completed. It is created as Pending and stays hidden from trekkers until the admin approves it, only an Open trek can be booked, and the assigned guide moves it to Closed or Completed.

Database

The database file is created automatically at `instance/trek.db` on the first run. The default admin account is `admin` with password `admin123` and it is created only once.

Video Link

<add the unlisted video link here before submitting>

AI Declaration

I have used AI/LLM assistance for this project and I am declaring the extent of that use below.

Tool used: Claude, an AI coding assistant.

What was AI assisted: the Flask route code in `app.py` , the database models in `models.py` , the Jinja2 templates, the stylesheet, the sample data used for the demonstration, and the first drafts of this report and the project README.

What I did myself: reading the problem statement and deciding the requirements, checking the application against every requirement in the problem statement, identifying the features that were missing or did not match the specification and getting them corrected, testing the screens for each role, and the final review of the submitted code.

Understanding of the code: I have gone through the complete codebase and I am able to explain the routing, the role checks in every protected route, the slot handling that prevents overbooking, and the relationships between the User, Trek and Booking tables. I am able to modify any part of the application during the viva.